

Memoria Práctica 1: Simulación del Dogma Central de la Biología Molecular

Introducción y Metodología

El objetivo fundamental de esta práctica es implementar un simulador bioinformático que represente el flujo completo de la información genética ($ADN \rightarrow ADN$, $ADN \rightarrow ARN$ y $ARN \rightarrow Proteína$), analizando además las implicaciones estructurales y funcionales de los procesos de replicación, transcripción, traducción y splicing alternativo.

Metodología y Entorno

- **Lenguaje y versión:** Python 3.11+.
- **Librerías principales:** Biopython (Bio.Seq, Bio.PDB), requests (API REST de Ensembl).
- **Control de versiones:** Git & GitHub.

Ejercicio 1. Replicación del ADN

Dada la secuencia original de ADN (5'-ATG CCG TTA GCT-3'/3'-TAC GGC AAT CGA-5'):

- **Molécula Replicada 1:** Hebra molde (3' → 5'): 3'-TAC GGC AAT CGA-5' | Hebra nueva (5' → 3'): 5'-ATG CCG TTA GCT-3'
- **Molécula Replicada 2:** Hebra molde (5' → 3'): 5'-ATG CCG TTA GCT-3' | Hebra nueva (3' → 5'): 3'-TAC GGC AAT CGA-5'

Enzimas Implicadas

- **Helicasa:** Separa las dos hebras de la doble hélice rompiendo los puentes de hidrógeno.
- **Primasa:** Sintetiza cebadores de ARN (primers) para proporcionar el extremo 3'-OH libre.
- **ADN Polimerasa:** Incorpora desoxirribonucleótidos en sentido 5' → 3' por complementariedad de bases.
- **Ligasa:** Une los fragmentos de Okazaki mediante enlaces fosfodiéster en la cadena rezagada.

Reflexión: Errores de la ADN Polimerasa

Un error no corregido fija una **mutación puntual** permanente. En regiones codificantes, esto puede alterar el aminoácido resultante (mutación de sentido erróneo) o generar un codón de paro prematuro (mutación sin sentido).

Ejercicio 2. Transcripción del ADN a ARN

Dada la secuencia 5'-ATG CCT GAA TGC-3'/3'-TAC GGA CTT ACG-5':

- **Cadena Molde (3' → 5'):** 3'-TAC GGA CTT ACG-5'
- **Transcrito de ARNm (5' → 3'):** 5'-AUG CCU GAA UGC-3'

Región Promotora vs. Región Codificante

- **Promotor:** Región de reconocimiento del ADN donde se unen la ARN polimerasa y factores de transcripción. No se transcribe a proteína.
- **Región Codificante:** Tramo de nucleótidos que se transcriben de forma efectiva a ARNm para su posterior traducción.

Ejercicio 3. Traducción del ARNm a Proteína

Dado el transcrito de ARNm 5'-AUG UAU GCU UAA-3':

- **Codón de Inicio:** AUG (Metionina)
- **Codón de Paro:** UAA (STOP)
- **Secuencia Traducida:** Met – Tyr – Ala

Reflexión: Mutaciones en Codones de Inicio y Paro

- **Mutación AUG → GUG:** Dificulta el reconocimiento canónico por el ribosoma, reduciendo drásticamente la tasa de inicio de traducción.
- **Pérdida del codón de paro:** El ribosoma continuará leyendo la región 3' UTR hasta hallar un codón STOP fuera de marco, produciendo una proteína anómala e inestable.

Ejercicio 4. Splicing Alternativo

Para un gen formado por los exones 1 – 2 – 3 – 4 – 5, se han diseñado dos isoformas:

- **Isoforma A (1 – 2 – 4 – 5):** Exclusión del Exón 3.
- **Isoforma B (1 – 3 – 5):** Exclusión de los Exones 2 y 4.

Análisis del Gen Human FGFR2 en Ensembl

A través del API REST de Ensembl se consultó el gen FGFR2 (ENSG00000066468), identificando un total de 59 transcritos.

Isoforma	Exón Clave	Expresión y Función Biológica
FGFR2b	Exón IIIb	Tejido epitelial (piel) – Desarrollo de la epidermis.
FGFR2c	Exón IIIc	Tejido mesenquimático – Osteogénesis (huesos).

Ejercicio 5. Introducción a las Proteínas

Dada la cadena peptídica Met – Ile – Ser – Gly – Val – Lys – His:

- **Extremo N-terminal:** Metionina (Met) con grupo amino (-NH₂) libre.
- **Extremo C-terminal:** Histidina (His) con grupo carboxilo (-COOH) libre.

Exploración Estructural en el PDB (1A6M)

Mediante el módulo Bio.PDB de Biopython, se analizó la estructura 3D de la mioglobina (1A6M), identificando 151 residuos de aminoácidos organizados predominantemente en hélices α .

```
1 from Bio.PDB import PDBList, MMCIFParser
2 pdbl = PDBList()
3 file_path = pdbl.retrieve_pdb_file("1A6M", pdir=".", file_format="mmCif"
4 )
5 parser = MMCIFParser(QUIET=True)
6 structure = parser.get_structure("1A6M", file_path)
7 residues = [r for r in structure.get_residues() if r.get_id()[0] == ' ']
8 print(f"Total aminoacidos: {len(residues)}")
```

Ejercicio 6. Actividad Integradora: Pipeline del Dogma Central

Se desarrolló un pipeline completo en Python que toma como entrada el gen real lacZ extraído del paquete NCBI (gene.fna).

Reflexión: Punto más vulnerable a errores

El proceso más crítico es la **Replicación del ADN**. Las mutaciones genómicas son permanentes y hereditarias para todas las divisiones celulares. Un error en la transcripción o traducción solo afecta a moléculas individuales transitorias que la célula puede degradar.

Conclusiones

La simulación del dogma central permite constatar cómo la información genética se transmite y procesa con alta fidelidad a través de los mecanismos moleculares. Herramientas bioinformáticas como Biopython y las APIs públicas (Ensembl, PDB) resultan indispensables para automatizar la traducción de secuencias, analizar el splicing alternativo y relacionar la estructura tridimensional con la función proteica.